

```
In [1]: import numpy as np
```

create numpy Array

```
In [2]: arr=np.array([1,2,3,4,5,6])          # 1d array
arr
```

```
Out[2]: array([1, 2, 3, 4, 5, 6])
```

```
In [3]: print(arr)
```

```
[1 2 3 4 5 6]
```

```
In [5]: type(arr)
```

```
Out[5]: numpy.ndarray
```

```
In [7]: arr2=np.array([[1 , 2 , 3 , 4],[ 5 , 6 , 7 , 8]])      # 2d array
arr2
```

```
Out[7]: array([[1, 2, 3, 4],
               [5, 6, 7, 8]])
```

```
In [10]: arr3=np.array([[ 2, 3 , 5,6],[4, 5, 6, 7],[24,71, 56,45],[45,76,82,35]])
arr3
```

```
Out[10]: array([[ 2,  3,  5,  6],
                [ 4,  5,  6,  7],
                [24, 71, 56, 45],
                [45, 76, 82, 35]])
```

dtype

```
In [11]: np.array([1,2,3,4],dtype=int)
```

```
Out[11]: array([1, 2, 3, 4])
```

```
In [12]: np.array([1,2,3,4],dtype=float)
```

```
Out[12]: array([1., 2., 3., 4.])
```

```
In [13]: np.array([1,2,3,4],dtype=complex)
```

```
Out[13]: array([1.+0.j, 2.+0.j, 3.+0.j, 4.+0.j])
```

```
In [14]: np.array([1,2,3,4],dtype=bool)
```

```
Out[14]: array([ True,  True,  True,  True])
```

arange

```
In [16]: np.arange(1,13)           # it works as range
```

```
Out[16]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12])
```

```
In [17]: np.arange(1,14,2)        # 1-13 , step 2
```

```
Out[17]: array([ 1,  3,  5,  7,  9, 11, 13])
```

reshape

```
In [18]: np.arange(1,13).reshape(2,6)      # 2 rows, 6 column
```

```
Out[18]: array([[ 1,  2,  3,  4,  5,  6],
                [ 7,  8,  9, 10, 11, 12]])
```

```
In [19]: np.arange(1,13).reshape(6,2)      # 6 rows, 2 column
```

```
Out[19]: array([[ 1,  2],
                [ 3,  4],
                [ 5,  6],
                [ 7,  8],
                [ 9, 10],
                [11, 12]])
```

```
In [20]: np.arange(1,13).reshape(3,4)      # 3 rows, 4 column
```

```
Out[20]: array([[ 1,  2,  3,  4],
                [ 5,  6,  7,  8],
                [ 9, 10, 11, 12]])
```

ones & zeros

```
In [21]: np.ones((3,4))           # bydefault in float
```

```
Out[21]: array([[1., 1., 1., 1.],
                [1., 1., 1., 1.],
                [1., 1., 1., 1.]])
```

```
In [24]: np.ones((3,4),dtype=int)
```

```
Out[24]: array([[1, 1, 1, 1],
                [1, 1, 1, 1],
                [1, 1, 1, 1]])
```

```
In [25]: np.zeros((6,3))         # bydefault in float
```

```
Out[25]: array([[0., 0., 0.],
                [0., 0., 0.],
                [0., 0., 0.],
                [0., 0., 0.],
                [0., 0., 0.],
                [0., 0., 0.]])
```

```
In [26]: np.zeros((6,3),dtype=int)
```

```
Out[26]: array([[0, 0, 0],
               [0, 0, 0],
               [0, 0, 0],
               [0, 0, 0],
               [0, 0, 0],
               [0, 0, 0]])
```

random,randoint - value range betn 0-1

```
In [27]: np.random.rand(4)           # bydefault in float
```

```
Out[27]: array([0.49030379, 0.65090594, 0.55704383, 0.72085301])
```

```
In [28]: np.random.randint(1,14)    # random no.
```

```
Out[28]: 10
```

```
In [29]: np.random.randint(1,14,5)  #, random 1-13 , 5 no.
```

```
Out[29]: array([ 3,  7, 11, 10,  4], dtype=int32)
```

linspace - same difference of gap in each value

```
In [31]: np.linspace(1,13,4)
```

```
Out[31]: array([ 1.,  5.,  9., 13.])
```

```
In [33]: np.linspace(-2,13,3)
```

```
Out[33]: array([-2. ,  5.5, 13. ])
```

identity

```
In [34]: np.identity(4)             # diagonal elem 1 , remaining value 0
```

```
Out[34]: array([[1., 0., 0., 0.],
               [0., 1., 0., 0.],
               [0., 0., 1., 0.],
               [0., 0., 0., 1.]])
```

```
In [35]: np.identity(3)             # 3 * 3 matrix
```

```
Out[35]: array([[1., 0., 0.],
               [0., 1., 0.],
               [0., 0., 1.]])
```

array attributes

```
In [37]: a1=np.arange(7)      # 1d  
a1
```

```
Out[37]: array([0, 1, 2, 3, 4, 5, 6])
```

```
In [40]: a2=np.arange(20,dtype=float).reshape(4,5)  
a2
```

```
Out[40]: array([[ 0.,  1.,  2.,  3.,  4.],  
                [ 5.,  6.,  7.,  8.,  9.],  
                [10., 11., 12., 13., 14.],  
                [15., 16., 17., 18., 19.]])
```

```
In [41]: a3=np.arange(27,dtype=float).reshape(3,3,3)  # 3d tensor  
a3
```

```
Out[41]: array([[[ 0.,  1.,  2.],  
                [ 3.,  4.,  5.],  
                [ 6.,  7.,  8.]],  
                 
               [[ 9., 10., 11.],  
                [12., 13., 14.],  
                [15., 16., 17.]],  
                 
               [[18., 19., 20.],  
                [21., 22., 23.],  
                [24., 25., 26.]])
```

ndim (n dimension)

```
In [42]: a1.ndim
```

```
Out[42]: 1
```

```
In [43]: a2.ndim
```

```
Out[43]: 2
```

```
In [44]: a3.ndim
```

```
Out[44]: 3
```

shape

```
In [45]: a1.shape
```

```
Out[45]: (7,)
```

```
In [46]: a2.shape
```

```
Out[46]: (4, 5)
```

```
In [47]: a3.shape
```



```
Out[47]: (3, 3, 3)
```

size - total elem

```
In [49]: a3
```

```
Out[49]: array([[ 0.,  1.,  2.],
                [ 3.,  4.,  5.],
                [ 6.,  7.,  8.]],

              [[ 9., 10., 11.],
               [12., 13., 14.],
               [15., 16., 17.]],

              [[18., 19., 20.],
               [21., 22., 23.],
               [24., 25., 26.]])
```

```
In [50]: a3.size
```

```
Out[50]: 27
```

```
In [51]: a2
```

```
Out[51]: array([[ 0.,  1.,  2.,  3.,  4.],
                [ 5.,  6.,  7.,  8.,  9.],
                [10., 11., 12., 13., 14.],
                [15., 16., 17., 18., 19.]])
```

```
In [52]: a2.size
```

```
Out[52]: 20
```

```
In [53]: a1
```

```
Out[53]: array([0, 1, 2, 3, 4, 5, 6])
```

```
In [54]: a1.size
```

```
Out[54]: 7
```

itemsize - result in bytes

64 bits = 8 bytes

```
In [55]: a1
```

```
Out[55]: array([0, 1, 2, 3, 4, 5, 6])
```

```
In [56]: a1.itemsize      #int64
```

Out[56]: 8

In [57]: a2

Out[57]: array([[0., 1., 2., 3., 4.],
[5., 6., 7., 8., 9.],
[10., 11., 12., 13., 14.],
[15., 16., 17., 18., 19.]])

In [59]: a2.itemsize *# float64*

Out[59]: 8

In [60]: a3

Out[60]: array([[[0., 1., 2.],
[3., 4., 5.],
[6., 7., 8.]],

[[9., 10., 11.],
[12., 13., 14.],
[15., 16., 17.]],

[[18., 19., 20.],
[21., 22., 23.],
[24., 25., 26.]])

In [61]: a3.itemsize *# float64*

Out[61]: 8

dtype

In [62]: print(a1.dtype)
print(a2.dtype)
print(a3.dtype)

int64
float64
float64

changing datatype - astype()

In [63]: a

Out[63]: array([0, 1, 2, 3, 4, 5, 6])

In [64]: a.astype(float)

Out[64]: array([0., 1., 2., 3., 4., 5., 6.])

array operation

```
In [66]: x1=np.arange(1,13).reshape(3,4)
x2=np.arange(12,24).reshape(4,3)
```

```
In [67]: x1
```

```
Out[67]: array([[ 1,  2,  3,  4],
                [ 5,  6,  7,  8],
                [ 9, 10, 11, 12]])
```

```
In [68]: x2
```

```
Out[68]: array([[12, 13, 14],
                [15, 16, 17],
                [18, 19, 20],
                [21, 22, 23]])
```

scalar operations

arithmetic operators

```
In [69]: x1
```

```
Out[69]: array([[ 1,  2,  3,  4],
                [ 5,  6,  7,  8],
                [ 9, 10, 11, 12]])
```

```
In [70]: x1 + 2
```

```
Out[70]: array([[ 3,  4,  5,  6],
                [ 7,  8,  9, 10],
                [11, 12, 13, 14]])
```

```
In [71]: x1 - 2
```

```
Out[71]: array([[ -1,  0,  1,  2],
                [  3,  4,  5,  6],
                [  7,  8,  9, 10]])
```

```
In [72]: x1 * 2
```

```
Out[72]: array([[ 2,  4,  6,  8],
                [10, 12, 14, 16],
                [18, 20, 22, 24]])
```

```
In [73]: x1 ** 2
```

```
Out[73]: array([[ 1,  4,  9, 16],
                [25, 36, 49, 64],
                [81, 100, 121, 144]])
```

```
In [74]: x1 % 2
```

```
Out[74]: array([[1, 0, 1, 0],
                [1, 0, 1, 0],
                [1, 0, 1, 0]])
```

```
In [75]: x1 // 2
```

```
Out[75]: array([[0, 1, 1, 2],
               [2, 3, 3, 4],
               [4, 5, 5, 6]])
```

relational operators

```
In [77]: x2
```

```
Out[77]: array([[12, 13, 14],
               [15, 16, 17],
               [18, 19, 20],
               [21, 22, 23]])
```

```
In [79]: x2 < 20
```

```
Out[79]: array([[ True,  True,  True],
               [ True,  True,  True],
               [ True,  True, False],
               [False, False, False]])
```

```
In [80]: x2 >= 15
```

```
Out[80]: array([[False, False, False],
               [ True,  True,  True],
               [ True,  True,  True],
               [ True,  True,  True]])
```

vector operations

```
In [81]: x1
```

```
Out[81]: array([[ 1,  2,  3,  4],
               [ 5,  6,  7,  8],
               [ 9, 10, 11, 12]])
```

```
In [82]: x2
```

```
Out[82]: array([[12, 13, 14],
               [15, 16, 17],
               [18, 19, 20],
               [21, 22, 23]])
```

```
In [84]: x1 + x2      # shape must be same
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[84], line 1
----> 1 x1 + x2

ValueError: operands could not be broadcast together with shapes (3,4) (4,3)
```

```
In [88]: z1=np.arange(1,11).reshape(2,5)
z1
```

```
Out[88]: array([[ 1,  2,  3,  4,  5],  
               [ 6,  7,  8,  9, 10]])
```

```
In [89]: z2=np.arange(11,21).reshape(2,5)  
z2
```

```
Out[89]: array([[11, 12, 13, 14, 15],  
               [16, 17, 18, 19, 20]])
```

```
In [90]: z1 + z2
```

```
Out[90]: array([[12, 14, 16, 18, 20],  
               [22, 24, 26, 28, 30]])
```

```
In [91]: z1 - z2
```

```
Out[91]: array([[ -10, -10, -10, -10, -10],  
               [-10, -10, -10, -10, -10]])
```

```
In [92]: z1 * z2
```

```
Out[92]: array([[ 11,  24,  39,  56,  75],  
               [ 96, 119, 144, 171, 200]])
```

```
In [93]: z1 / z2
```

```
Out[93]: array([[0.09090909, 0.16666667, 0.23076923, 0.28571429, 0.33333333],  
               [0.375      , 0.41176471, 0.44444444, 0.47368421, 0.5       ]])
```

```
In [94]: z1//z2
```

```
Out[94]: array([[0, 0, 0, 0, 0],  
               [0, 0, 0, 0, 0]])
```

```
In [95]: z1%z2
```

```
Out[95]: array([[ 1,  2,  3,  4,  5],  
               [ 6,  7,  8,  9, 10]])
```

array function

```
In [96]: k1 = np.random.random((3,3))  
k1
```

```
Out[96]: array([[0.51405708, 0.98088702, 0.90910516],  
               [0.15431119, 0.88803197, 0.23695835],  
               [0.98722035, 0.20482784, 0.56044665]])
```

```
In [97]: k1 = np.round(k1*100)  
k1
```

```
Out[97]: array([[51., 98., 91.],  
               [15., 89., 24.],  
               [99., 20., 56.]])
```

```
In [98]: np.max(k1)
```

```
Out[98]: np.float64(99.0)
```

```
In [99]: np.min(k1)
```

```
Out[99]: np.float64(15.0)
```

```
In [100]: np.sum(k1)
```

```
Out[100]: np.float64(543.0)
```

```
In [101]: np.prod(k1)
```

```
Out[101]: np.float64(1615784243673600.0)
```

axis = 0 - column

axis = 1 - row

```
In [105]: k1
```

```
Out[105]: array([[51., 98., 91.],  
                [15., 89., 24.],  
                [99., 20., 56.]])
```

```
In [106]: np.max(k1,axis = 0)
```

```
Out[106]: array([99., 98., 91.])
```

```
In [108]: np.max(k1,axis = 1)
```

```
Out[108]: array([98., 89., 99.])
```

```
In [109]: np.min(k1,axis = 0)
```

```
Out[109]: array([15., 20., 24.])
```

```
In [111]: np.min(k1,axis = 1)
```

```
Out[111]: array([51., 15., 20.])
```

```
In [112]: np.sum(k1,axis = 0)
```

```
Out[112]: array([165., 207., 171.])
```

```
In [113]: np.sum(k1,axis = 1)
```

```
Out[113]: array([240., 128., 175.])
```

```
In [114]: np.prod(k1,axis = 0)
```

```
Out[114]: array([ 75735., 174440., 122304.])
```

```
In [115... np.prod(k1,axis = 1)

Out[115... array([454818., 32040., 110880.])
```

statitics related functions

mean

```
In [116... k1

Out[116... array([[51., 98., 91.],
               [15., 89., 24.],
               [99., 20., 56.]])

In [117... np.mean(k1)

Out[117... np.float64(60.333333333333336)

In [118... np.mean(k1,axis=0)

Out[118... array([55., 69., 57.])

In [119... np.mean(k1,axis=1)

Out[119... array([80.          , 42.66666667, 58.33333333])
```

median

```
In [120... k1

Out[120... array([[51., 98., 91.],
               [15., 89., 24.],
               [99., 20., 56.]])

In [121... np.median(k1,axis=0)

Out[121... array([51., 89., 56.])

In [122... np.median(k1,axis=1)

Out[122... array([91., 24., 56.])
```

variance

```
In [124... np.var(k1,axis=0)

Out[124... array([1184.          , 1214.          , 748.66666667])

In [125... np.var(k1,axis=1)
```

```
Out[125...] array([ 428.66666667, 1086.88888889, 1042.88888889])
```

dot product

```
In [131...] t1=np.arange(1,11).reshape(2,5)
            t2=np.arange(11,21).reshape(5,2)
```

```
In [132...] print(t1)
```

```
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]]
```

```
In [133...] print(t2)
```

```
[[11 12]
 [13 14]
 [15 16]
 [17 18]
 [19 20]]
```

```
In [134...] np.dot(t1,t2)      # mult of row and column
```

```
Out[134...] array([[245, 260],
                  [620, 660]])
```

round,floor,ceil

```
In [139...] f1=np.array([1.3421312 , 2.76834927 ,3.6492888 , 4.437923810 , 5.89013810,])
```

```
In [140...] np.round(f1,decimals=2)
```

```
Out[140...] array([1.34, 2.77, 3.65, 4.44, 5.89])
```

```
In [141...] np.floor(f1)      # approx Lower value
```

```
Out[141...] array([1., 2., 3., 4., 5.])
```

```
In [142...] np.ceil(f1)      # approx upper value
```

```
Out[142...] array([2., 3., 4., 5., 6.])
```

indexing and slicing

```
In [167...] j1=np.arange(4,10)
            j2=np.arange(1,7).reshape(2,3)
            j3=np.arange(27).reshape(3,3,3)
```

```
In [168...] j1
```

```
Out[168...] array([4, 5, 6, 7, 8, 9])
```

```
In [169...] j2
```



```
Out[169... array([[1, 2, 3],  
          [4, 5, 6]])
```

```
In [170... j3
```

```
Out[170... array([[[ 0, 1, 2],  
          [ 3, 4, 5],  
          [ 6, 7, 8]],  
  
          [[ 9, 10, 11],  
          [12, 13, 14],  
          [15, 16, 17]],  
  
          [[18, 19, 20],  
          [21, 22, 23],  
          [24, 25, 26]]])
```

```
In [171... j3[1]
```

```
Out[171... array([[ 9, 10, 11],  
          [12, 13, 14],  
          [15, 16, 17]])
```

```
In [172... j1 # 1d
```

```
Out[172... array([4, 5, 6, 7, 8, 9])
```

```
In [175... print(j1[2])
```

6

```
In [176... print(j1[-2])
```

8

```
In [177... j2 # 2d
```

```
Out[177... array([[1, 2, 3],  
          [4, 5, 6]])
```

```
In [180... print(j2[0,1])
```

2

```
In [181... print(j2[0,2])
```

3

```
In [182... j3 # 3d
```

```
Out[182... array([[[ 0, 1, 2],  
          [ 3, 4, 5],  
          [ 6, 7, 8]],  
  
          [[ 9, 10, 11],  
          [12, 13, 14],  
          [15, 16, 17]],  
  
          [[18, 19, 20],  
          [21, 22, 23],  
          [24, 25, 26]]])
```

```
In [183... print(j3[2,1,2])      # 3rd block, 2nd row, 3rd col
```

23

```
In [184... print(j3[1,0,1])
```

10

```
In [185... print(j3[0,0,1])
```

1

```
In [186... print(j3[0,0,0])
```

0

slicing

slicing on 1d

```
In [187... j1
```

```
Out[187... array([4, 5, 6, 7, 8, 9])
```

```
In [189... print(j1[1:4])
```

[5 6 7]

```
In [191... print(j1[0:-4])
```

[4 5]

```
In [192... print(j1[:])
```

[4 5 6 7 8 9]

slicing on 2d

```
In [193... j2
```

```
Out[193... array([[1, 2, 3],  
                [4, 5, 6]])
```

```
In [194... print(j2[1:4])
```

[[4 5 6]]

```
In [197... print(j2[0:, -3:])
```

[[1 2 3]
 [4 5 6]]

```
In [210... print(j2[1:3, -4: :])
```

[[4 5 6]]

slicing on 3d

In [199...

j3

Out[199...

```
array([[[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8]],

       [[ 9, 10, 11],
        [12, 13, 14],
        [15, 16, 17]],

       [[18, 19, 20],
        [21, 22, 23],
        [24, 25, 26]])
```

In [202...

```
print(j3[0:2 , 1:3 ,1:3])
```

```
[[[ 4  5]
   [ 7  8]]
```

```
[[13 14]
 [16 17]]]
```

In [206...

```
print(j3[1:3 , 1:5 ,-5:-2])
```

```
[[[12]
   [15]]
```

```
[[21]
 [24]]]
```

In [208...

```
print(j3[1:3 , 1:5 ,: :-2])
```

```
[[[14 12]
   [17 15]]
```

```
[[23 21]
 [26 24]]]
```

In [209...

```
print(j3[1:3 , 1: : 2 ,: :-2])
```

```
[[[14 12]]
```

```
[[23 21]]]
```

iterating

looping array in 1d

In [211...

j1

Out[211...

```
array([4, 5, 6, 7, 8, 9])
```

```
In [212... for i in j1:
             print(i)
```

```
4
5
6
7
8
9
```

looping array in 2d

```
In [213... j2
```

```
Out[213... array([[1, 2, 3],
          [4, 5, 6]])
```

```
In [215... for i in j2:
             print(i)
```

```
[1 2 3]
[4 5 6]
```

looping array in 3d

```
In [216... j3
```

```
Out[216... array([[[ 0,  1,  2],
          [ 3,  4,  5],
          [ 6,  7,  8]],

          [[ 9, 10, 11],
          [12, 13, 14],
          [15, 16, 17]],

          [[18, 19, 20],
          [21, 22, 23],
          [24, 25, 26]]])
```

```
In [217... for i in j3:
             print(i)
```

```
[[0 1 2]
 [3 4 5]
 [6 7 8]]
[[ 9 10 11]
 [12 13 14]
 [15 16 17]]
[[18 19 20]
 [21 22 23]
 [24 25 26]]
```

```
In [218... for i in np.nditer(j3):
             print(i)
```

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26

transpose

In [219...

j1

Out[219... array([4, 5, 6, 7, 8, 9])

In [220... np.transpose(j1) *# 1d transpose not possible bcoz there is no col and rows*

Out[220... array([4, 5, 6, 7, 8, 9])

In [221... j2

Out[221... array([[1, 2, 3],
[4, 5, 6]])

In [222... np.transpose(j2)

Out[222... array([[1, 4],
[2, 5],
[3, 6]])

In [223... j3

```
Out[223...] array([[ 0,  1,  2],
               [ 3,  4,  5],
               [ 6,  7,  8]],

               [[ 9, 10, 11],
               [12, 13, 14],
               [15, 16, 17]],

               [[18, 19, 20],
               [21, 22, 23],
               [24, 25, 26]])
```

```
In [224...] np.transpose(j3)
```

```
Out[224...] array([[[ 0,  9, 18],
                    [ 3, 12, 21],
                    [ 6, 15, 24]],

                    [[ 1, 10, 19],
                    [ 4, 13, 22],
                    [ 7, 16, 25]],

                    [[ 2, 11, 20],
                    [ 5, 14, 23],
                    [ 8, 17, 26]])
```

```
In [225...] j2
```

```
Out[225...] array([[1, 2, 3],
                   [4, 5, 6]])
```

```
In [226...] j2.T      # 2d
```

```
Out[226...] array([[1, 4],
                   [2, 5],
                   [3, 6]])
```

```
In [227...] j3
```

```
Out[227...] array([[[ 0,  1,  2],
                    [ 3,  4,  5],
                    [ 6,  7,  8]],

                    [[ 9, 10, 11],
                    [12, 13, 14],
                    [15, 16, 17]],

                    [[18, 19, 20],
                    [21, 22, 23],
                    [24, 25, 26]])
```

```
In [228...] j3.T      # 3d
```

```
Out[228...] array([[ 0,  9, 18],
          [ 3, 12, 21],
          [ 6, 15, 24]],

          [[ 1, 10, 19],
          [ 4, 13, 22],
          [ 7, 16, 25]],

          [[ 2, 11, 20],
          [ 5, 14, 23],
          [ 8, 17, 26]])
```

ravel

```
In [229...] # convert any dimension to 1d
```

```
In [230...] j2
```

```
Out[230...] array([[1, 2, 3],
          [4, 5, 6]])
```

```
In [232...] j2.ravel()
```

```
Out[232...] array([1, 2, 3, 4, 5, 6])
```

```
In [233...] j3
```

```
Out[233...] array([[[ 0,  1,  2],
          [ 3,  4,  5],
          [ 6,  7,  8]],

          [[ 9, 10, 11],
          [12, 13, 14],
          [15, 16, 17]],

          [[18, 19, 20],
          [21, 22, 23],
          [24, 25, 26]])
```

```
In [234...] j3.ravel()
```

```
Out[234...] array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
          17, 18, 19, 20, 21, 22, 23, 24, 25, 26])
```

```
In [235...] # stacking - dimension must be same
```

```
In [237...] w1=np.arange(0,12).reshape(3,4)
w2=np.arange(12,24).reshape(3,4)
```

```
In [238...] w1
```

```
Out[238...] array([[ 0,  1,  2,  3],
          [ 4,  5,  6,  7],
          [ 8,  9, 10, 11]])
```

```
In [239...] w2
```

```
Out[239...] array([[12, 13, 14, 15],  
          [16, 17, 18, 19],  
          [20, 21, 22, 23]])
```

```
In [241...] np.hstack((w1,w2))
```

```
Out[241...] array([[ 0,  1,  2,  3, 12, 13, 14, 15],  
          [ 4,  5,  6,  7, 16, 17, 18, 19],  
          [ 8,  9, 10, 11, 20, 21, 22, 23]])
```

```
In [242...] np.vstack((w1,w2))
```

```
Out[242...] array([[ 0,  1,  2,  3],  
          [ 4,  5,  6,  7],  
          [ 8,  9, 10, 11],  
          [12, 13, 14, 15],  
          [16, 17, 18, 19],  
          [20, 21, 22, 23]])
```

```
In [246...] # splitting - opposite of stacking
```

horizontal --> vertical and vertical --> horizontal and

```
In [247...] w1
```

```
Out[247...] array([[ 0,  1,  2,  3],  
          [ 4,  5,  6,  7],  
          [ 8,  9, 10, 11]])
```

```
In [248...] np.hsplit(w1,2) # split in 2 blocks
```

```
Out[248...] [array([[0, 1],  
          [4, 5],  
          [8, 9]]),  
          array([[ 2,  3],  
          [ 6,  7],  
          [10, 11]])]
```

```
In [249...] np.vsplit(w1,3) # split in 3 blocks
```

```
Out[249...] [array([[0, 1, 2, 3]]), array([[4, 5, 6, 7]]), array([[ 8,  9, 10, 11]])]
```

```
In [250...] w2
```

```
Out[250...] array([[12, 13, 14, 15],  
          [16, 17, 18, 19],  
          [20, 21, 22, 23]])
```

```
In [251...] np.hsplit(w2,4)
```



```
Out[251...] [array([[12],
        [16],
        [20]]),
array([[13],
        [17],
        [21]]),
array([[14],
        [18],
        [22]]),
array([[15],
        [19],
        [23]])]
```

```
In [254...] np.vsplit(w2,3)
```

```
Out[254...] [array([[12, 13, 14, 15]]),
array([[16, 17, 18, 19]]),
array([[20, 21, 22, 23]])]
```

advanced indexing and slicing

```
In [255...] w=np.arange(12).reshape(3,4)
w
```

```
Out[255...] array([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

```
In [259...] print(w[1,3] )           # 1th row , 3rd col
```

7

```
In [260...] w[1:4]           # 1th row to 3rd row slicing
```

```
Out[260...] array([[ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

```
In [257...] w[0:5]           # 0th row to 4th row slicing
```

```
Out[257...] array([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

```
In [262...] w
```

```
Out[262...] array([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

```
In [263...] w[1:3,2:4]
```

```
Out[263...] array([[ 6,  7],
        [10, 11]])
```

fancy indexing

In [264...

w

Out[264...

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

row

In [266...

```
w[[0,2]]          # 0 , 2   row
```

Out[266...

```
array([[ 0,  1,  2,  3],
       [ 8,  9, 10, 11]])
```

In [267...

```
w[[1,2]]
```

Out[267...

```
array([[ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [268...

```
w[[0,1,2]]
```

Out[268...

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

column

In [269...

```
b=np.arange(0,24).reshape(4,6)
b
```

Out[269...

```
array([[ 0,  1,  2,  3,  4,  5],
       [ 6,  7,  8,  9, 10, 11],
       [12, 13, 14, 15, 16, 17],
       [18, 19, 20, 21, 22, 23]])
```

In [271...

```
b[[0,2,3]]        # row
```

Out[271...

```
array([[ 0,  1,  2,  3,  4,  5],
       [12, 13, 14, 15, 16, 17],
       [18, 19, 20, 21, 22, 23]])
```

In [274...

```
b[:, [1,3,4]]     # 1th , 3rd , 4th column
```

Out[274...

```
array([[ 1,  3,  4],
       [ 7,  9, 10],
       [13, 15, 16],
       [19, 21, 22]])
```

In [275...

```
b[:, [2,4]]
```

Out[275...

```
array([[ 2,  4],
       [ 8, 10],
       [14, 16],
       [20, 22]])
```

Boolean indexing

```
In [283... G=np.random.randint(0,125,24).reshape(6,4)
G
```

```
Out[283... array([[ 8, 32, 85, 19],
        [ 92, 108, 26, 93],
        [ 8, 29, 84, 123],
        [ 18, 53, 0, 52],
        [ 95, 99, 25, 16],
        [ 30, 53, 103, 5]], dtype=int32)
```

```
In [284... G>50
```

```
Out[284... array([[False, False,  True, False],
        [ True,  True, False,  True],
        [False, False,  True,  True],
        [False,  True, False,  True],
        [ True,  True, False, False],
        [False,  True,  True, False]])
```

```
In [285... G[G>=30]
```

```
Out[285... array([ 32, 85, 92, 108, 93, 84, 123, 53, 52, 95, 99, 30, 53,
        103], dtype=int32)
```

```
In [286... G%3
```

```
Out[286... array([[2, 2, 1, 1],
        [2, 0, 2, 0],
        [2, 2, 0, 0],
        [0, 2, 0, 1],
        [2, 0, 1, 1],
        [0, 2, 1, 2]], dtype=int32)
```

```
In [288... G[G%3==0]
```

```
Out[288... array([108, 93, 84, 123, 18, 0, 99, 30], dtype=int32)
```

```
In [289... G [(G > 50) & (G % 2 == 0)]
```

```
Out[289... array([ 92, 108, 84, 52], dtype=int32)
```

Broadcasting

same shape

```
a=np.arange(0,6).reshape(2,3) b=np.arange(6,12).reshape(2,3) print(a) print(b) print(a+b)
```

diff shape

2d convert to 1d (1,3) or (3,1) 3d convert to 1d (1,1,3)

```
In [295... a=np.arange(0,6).reshape(2,3)
b=np.arange(3).reshape(1,3)
print(a)
print(b)

print(a+b)
```

```
[[0 1 2]
 [3 4 5]]
[[0 1 2]]
[[0 2 4]
 [3 5 7]]
```

np.sort()

```
In [296... a= np.random.randint(1,100,15) # 1d # 1-99 random 15 no.
a
```

```
Out[296... array([90, 10, 26, 61, 78,  9, 23, 43, 14, 62, 65, 52,  8, 42, 40],
      dtype=int32)
```

```
In [297... b=np.random.randint(1,100,18).reshape(3,6)
b
```

```
Out[297... array([[20, 55, 31, 24, 14, 49],
      [75, 33, 10, 92, 53, 62],
      [85, 41, 48, 42, 46, 83]], dtype=int32)
```

```
In [298... np.sort(a)
```

```
Out[298... array([ 8,  9, 10, 14, 23, 26, 40, 42, 43, 52, 61, 62, 65, 78, 90],
      dtype=int32)
```

```
In [299... np.sort(a)[::-1]
```

```
Out[299... array([90, 78, 65, 62, 61, 52, 43, 42, 40, 26, 23, 14, 10,  9,  8],
      dtype=int32)
```

```
In [300... b
```

```
Out[300... array([[20, 55, 31, 24, 14, 49],
      [75, 33, 10, 92, 53, 62],
      [85, 41, 48, 42, 46, 83]], dtype=int32)
```

```
In [301... np.sort(b,axis=1) # row wise
```

```
Out[301... array([[14, 20, 24, 31, 49, 55],
      [10, 33, 53, 62, 75, 92],
      [41, 42, 46, 48, 83, 85]], dtype=int32)
```

```
In [303... b
```

```
Out[303... array([[20, 55, 31, 24, 14, 49],
      [75, 33, 10, 92, 53, 62],
      [85, 41, 48, 42, 46, 83]], dtype=int32)
```

```
In [304... np.sort(b,axis=0) # column wise
```

```
Out[304...] array([[20, 33, 10, 24, 14, 49],
        [75, 41, 31, 42, 46, 62],
        [85, 55, 48, 92, 53, 83]], dtype=int32)
```

np.where

```
In [305...] a
```

```
Out[305...] array([90, 10, 26, 61, 78,  9, 23, 43, 14, 62, 65, 52,  8, 42, 40],
        dtype=int32)
```

```
In [307...] np.where(a>15)
```

```
Out[307...] (array([ 0,  2,  3,  4,  6,  7,  9, 10, 11, 13, 14]),)
```

```
In [308...] np.where(a>30,0,a)          # >30 replace with 0
```

```
Out[308...] array([ 0, 10, 26,  0,  0,  9, 23,  0, 14,  0,  0,  0,  8,  0,  0],
        dtype=int32)
```

```
In [310...] np.where(a>50,5,a)
```

```
Out[310...] array([ 5, 10, 26,  5,  5,  9, 23, 43, 14,  5,  5,  5,  8, 42, 40],
        dtype=int32)
```

np.argmax , argmin

index no.

```
In [311...] a          # 1d
```

```
Out[311...] array([90, 10, 26, 61, 78,  9, 23, 43, 14, 62, 65, 52,  8, 42, 40],
        dtype=int32)
```

```
In [313...] print(np.argmax (a))
```

0

```
In [319...] print(np.argmin (a) )    # 8 is on 12th index no.
```

12

```
In [320...] b          # 2d
```

```
Out[320...] array([[20, 55, 31, 24, 14, 49],
        [75, 33, 10, 92, 53, 62],
        [85, 41, 48, 42, 46, 83]], dtype=int32)
```

```
In [322...] np.argmax (b,axis=0)      # each col ,highest no. index position
```

```
Out[322...] array([2, 0, 2, 1, 1, 2])
```

```
In [323...] np.argmax (b,axis=1)      # each row ,highest no. index position
```

```
Out[323...] array([1, 3, 0])
```

np.cumsum

```
In [324... # adding current value with next value
```

```
In [325... a
```

```
Out[325... array([90, 10, 26, 61, 78, 9, 23, 43, 14, 62, 65, 52, 8, 42, 40],  
      dtype=int32)
```

```
In [326... np.cumsum (a)
```

```
Out[326... array([ 90, 100, 126, 187, 265, 274, 297, 340, 354, 416, 481, 533, 541,  
      583, 623])
```

```
In [327... b
```

```
Out[327... array([[20, 55, 31, 24, 14, 49],  
      [75, 33, 10, 92, 53, 62],  
      [85, 41, 48, 42, 46, 83]], dtype=int32)
```

```
In [328... np.cumsum (b)
```

```
Out[328... array([ 20, 75, 106, 130, 144, 193, 268, 301, 311, 403, 456, 518, 603,  
      644, 692, 734, 780, 863])
```

```
In [329... np.cumsum (b,axis=0)
```

```
Out[329... array([[ 20, 55, 31, 24, 14, 49],  
      [ 95, 88, 41, 116, 67, 111],  
      [180, 129, 89, 158, 113, 194]])
```

```
In [330... np.cumsum (b,axis=1)
```

```
Out[330... array([[ 20, 75, 106, 130, 144, 193],  
      [ 75, 108, 118, 210, 263, 325],  
      [ 85, 126, 174, 216, 262, 345]])
```

np.percentile

```
In [331... a
```

```
Out[331... array([90, 10, 26, 61, 78, 9, 23, 43, 14, 62, 65, 52, 8, 42, 40],  
      dtype=int32)
```

```
In [333... print(np.percentile (a,100) )    # max
```

```
90.0
```

```
In [334... print(np.percentile (a,0) )    # min
```

```
8.0
```

```
In [335... print(np.percentile (a,50) )    # median
```

```
42.0
```

```
In [337... print(np.median(a))
```

```
42.0
```

np.isin

```
In [338... a
```

```
Out[338... array([90, 10, 26, 61, 78,  9, 23, 43, 14, 62, 65, 52,  8, 42, 40],
      dtype=int32)
```

```
In [339... items=[10,20,30,40,50,60,70]
np.isin(a,items)           # 10 and Last 40 is there that's why True
```

```
Out[339... array([False,  True, False, False, False, False, False, False, False,
      False, False, False, False, False,  True])
```

```
In [340... a[np.isin(a,items)]
```

```
Out[340... array([10, 40], dtype=int32)
```

```
In [341... # np.flip      - reverse the order
```

```
In [342... a
```

```
Out[342... array([90, 10, 26, 61, 78,  9, 23, 43, 14, 62, 65, 52,  8, 42, 40],
      dtype=int32)
```

```
In [344... np.flip(a)
```

```
Out[344... array([40, 42,  8, 52, 65, 62, 14, 43, 23,  9, 78, 61, 26, 10, 90],
      dtype=int32)
```

```
In [345... b
```

```
Out[345... array([[20, 55, 31, 24, 14, 49],
      [75, 33, 10, 92, 53, 62],
      [85, 41, 48, 42, 46, 83]], dtype=int32)
```

```
In [346... np.flip(b)
```

```
Out[346... array([[83, 46, 42, 48, 41, 85],
      [62, 53, 92, 10, 33, 75],
      [49, 14, 24, 31, 55, 20]], dtype=int32)
```

```
In [347... np.flip(b,axis=0)
```

```
Out[347... array([[85, 41, 48, 42, 46, 83],
      [75, 33, 10, 92, 53, 62],
      [20, 55, 31, 24, 14, 49]], dtype=int32)
```

```
In [348... np.flip(b,axis=1)
```

```
Out[348... array([[49, 14, 24, 31, 55, 20],
      [62, 53, 92, 10, 33, 75],
      [83, 46, 42, 48, 41, 85]], dtype=int32)
```

np.put

```
In [349... # specific elem index replace with given value
```

```
In [350... a
```

```
Out[350... array([90, 10, 26, 61, 78,  9, 23, 43, 14, 62, 65, 52,  8, 42, 40],
      dtype=int32)
```

```
In [356... np.put(a,[0,1],[110,200])      # [index],[value]
```

```
In [357... a
```

```
Out[357... array([110, 200, 26, 61, 78,  9, 23, 43, 14, 62, 65, 52,  8,
      42, 40], dtype=int32)
```

np.delete

```
In [358... a
```

```
Out[358... array([110, 200, 26, 61, 78,  9, 23, 43, 14, 62, 65, 52,  8,
      42, 40], dtype=int32)
```

```
In [359... np.delete(a,0)                # index no. value delete
```

```
Out[359... array([200, 26, 61, 78,  9, 23, 43, 14, 62, 65, 52,  8, 42,
      40], dtype=int32)
```

np.clip - limit the value

```
In [362... a
```

```
Out[362... array([110, 200, 26, 61, 78,  9, 23, 43, 14, 62, 65, 52,  8,
      42, 40], dtype=int32)
```

```
In [364... np.clip(a,a_min=15,a_max=50)      # 15 & below 15 replace with 15
      # 50 & above 50 replace with 50
```

```
Out[364... array([50, 50, 26, 50, 50, 15, 23, 43, 15, 50, 50, 50, 15, 42, 40],
      dtype=int32)
```

np.swapaxes

row to column column to row

```
In [366... arr=np.array([[1,2,3],[4,5,6]])
arr
```

```
Out[366... array([[1, 2, 3],
      [4, 5, 6]])
```

```
In [367... np.swapaxes(arr,0,1)
```



```
Out[367... array([[1, 4],  
                [2, 5],  
                [3, 6]])
```

```
In [ ]:
```